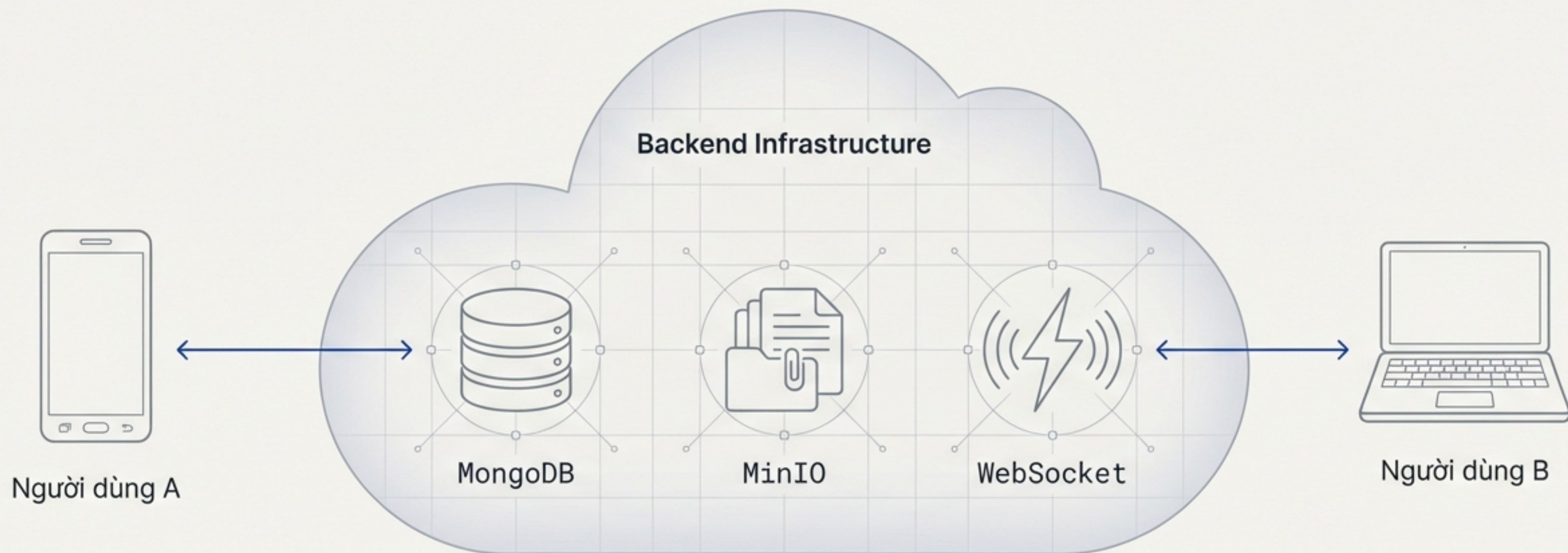


Kiến trúc Hệ thống Tin nhắn 1-1

Từ Nền tảng Dữ liệu đến Giao tiếp Thời gian thực



Tổng quan Kiến trúc Hệ thống

Hệ thống quản lý chat 1-1, bao gồm gửi tin nhắn, tệp đính kèm, reactions, và cài đặt hội thoại. Kiến trúc được xây dựng dựa trên ba trụ cột công nghệ chính để đảm bảo tính ổn định, hiệu quả và khả năng tương tác tức thì.



Giao tiếp Thời gian thực

Tận dụng WebSocket để cập nhật giao diện người dùng ngay lập tức cho các sự kiện như tin nhắn mới, trạng thái đã đọc và reactions.



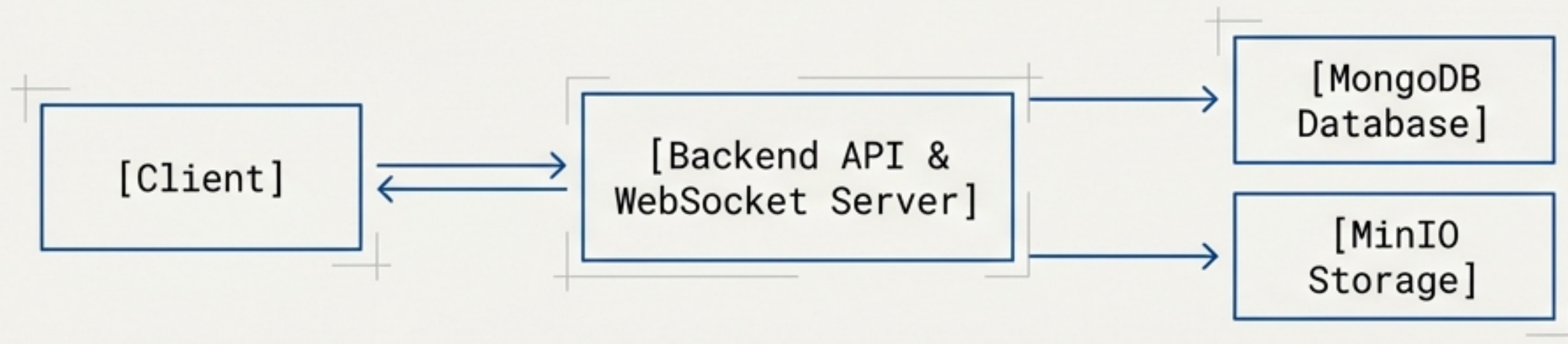
Lưu trữ Tệp Tích hợp

Sử dụng MinIO (S3-compatible) để quản lý việc tải lên và truy xuất tệp đính kèm một cách hiệu quả, tách biệt khỏi logic ứng dụng chính.



Quản lý Trạng thái Hội thoại

Một hệ thống cơ sở dữ liệu có cấu trúc tốt để theo dõi siêu dữ liệu hội thoại, số tin nhắn chưa đọc và cài đặt riêng của người dùng.



Nền tảng Dữ liệu (Phần 1): Model `Message`

Schema MongoDB cốt lõi, định nghĩa cấu trúc của mỗi tin nhắn. Thiết kế này ưu tiên tính toàn vẹn dữ liệu, khả năng truy vấn hiệu quả và các tính năng nâng cao.

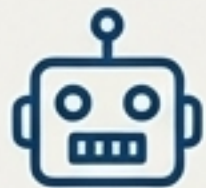
Các Quyết định Thiết kế Nổi bật



Soft Delete (`deletedBy`)

Thay vì xóa vật lý, tin nhắn được đánh dấu là đã xóa cho từng người dùng. Điều này bảo toàn lịch sử cho người dùng còn lại và giữ vững tính toàn vẹn tham chiếu.

`isDeletedFor(userId)` là một instance method để kiểm tra trạng thái này.



Tin nhắn Hệ thống (`isSystemMessage`)

Một cờ boolean để phân biệt tin nhắn do người dùng gửi và các thông báo tự động (ví dụ: "A đã ghim một tin nhắn"). Tin nhắn hệ thống không thể bị xóa bởi người dùng.



Theo dõi Trạng thái Đã đọc (`readBy`)

Một mảng ghi lại chính xác ai đã đọc tin nhắn và vào lúc nào, hỗ trợ cho các tính năng `seen by` chi tiết.



Định danh Hội thoại (`conversationId`)

Một ID nhất quán được tạo ra bằng cách sắp xếp ID của hai người dùng (`\${sortedId1}\_\${sortedId2}`). Đảm bảo cả hai người dùng đều truy cập vào cùng một luồng tin nhắn.

Chiến lược Indexing

Tối ưu hóa Truy vấn

```
{ conversationId: 1,
  createdAt: -1 }
```

Tối ưu hóa việc tải tin nhắn theo thứ tự thời gian và phân trang (pagination).

Tối ưu hóa Truy vấn

```
{ sender: 1, receiver: 1 }
```

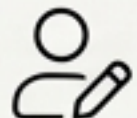
Tăng tốc các truy vấn liên quan đến người dùng cụ thể.

Nền tảng Dữ liệu (Phần 2): Model `Conversation`

Schema này hoạt động như một "bảng mục lục" cho tất cả các cuộc hội thoại, lưu trữ siêu dữ liệu và cài đặt. Việc tách biệt này giúp tải danh sách hội thoại cực kỳ nhanh chóng mà không cần truy vấn bộ sưu tập `Message` khổng lồ.

Các Cấu trúc Dữ liệu Thông minh

Sử dụng `Map` của Mongoose

	<code>unreadCount: `Map<UserId, Number>`</code> Cho phép truy cập và cập nhật số tin chưa đọc của mỗi người dùng một cách hiệu quả.
	<code>nicknames: `Map<UserId, String>`</code> Hỗ trợ đặt biệt danh tùy chỉnh cho mỗi người tham gia trong cuộc hội thoại.

Tối ưu hóa Sắp xếp

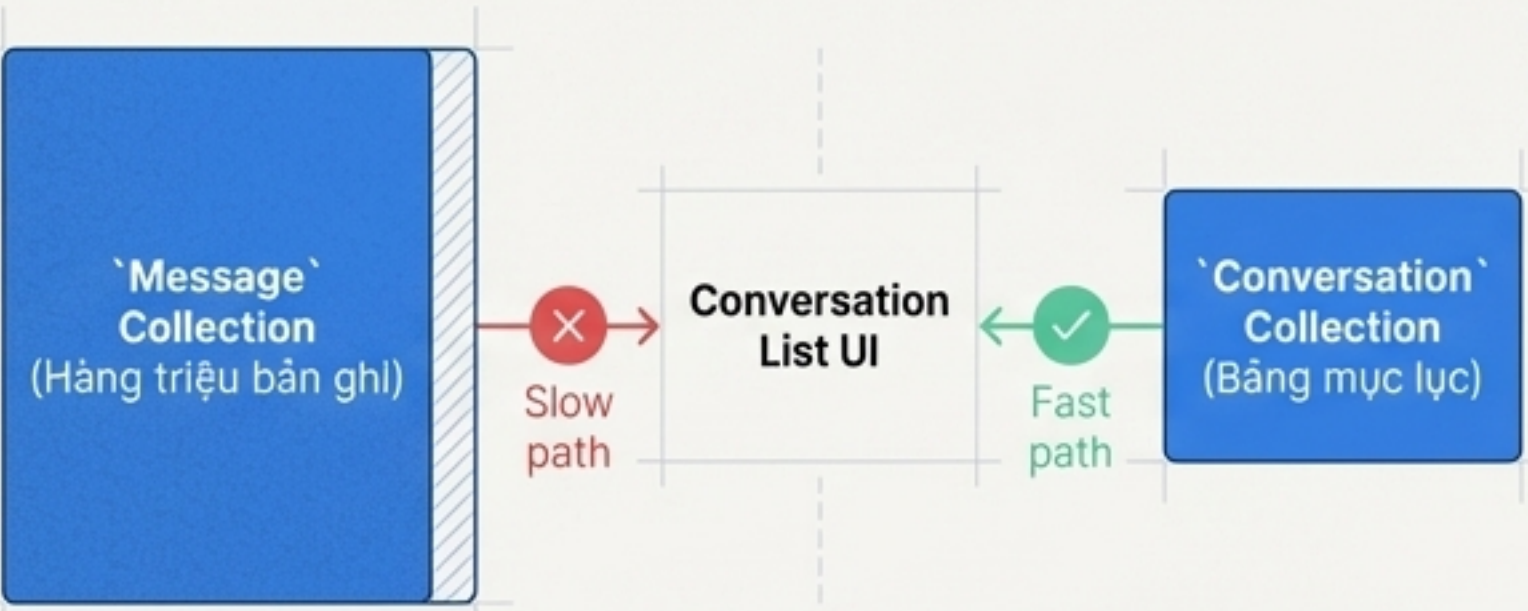
	<code>`lastMessageAt`</code> Một trường timestamp được cập nhật liên tục. Danh sách hội thoại chỉ cần sắp xếp theo trường này để luôn hiển thị các cuộc trò chuyện gần đây nhất lên đầu.
---	---

Indexing Chính & Visual

Chìa khóa Truy vấn Hiệu quả

```
{
  participants: 1,
  lastMessageAt: -1
}
```

Chìa khóa để truy vấn hiệu quả tất cả các cuộc hội thoại của một người dùng và sắp xếp chúng theo hoạt động gần đây.



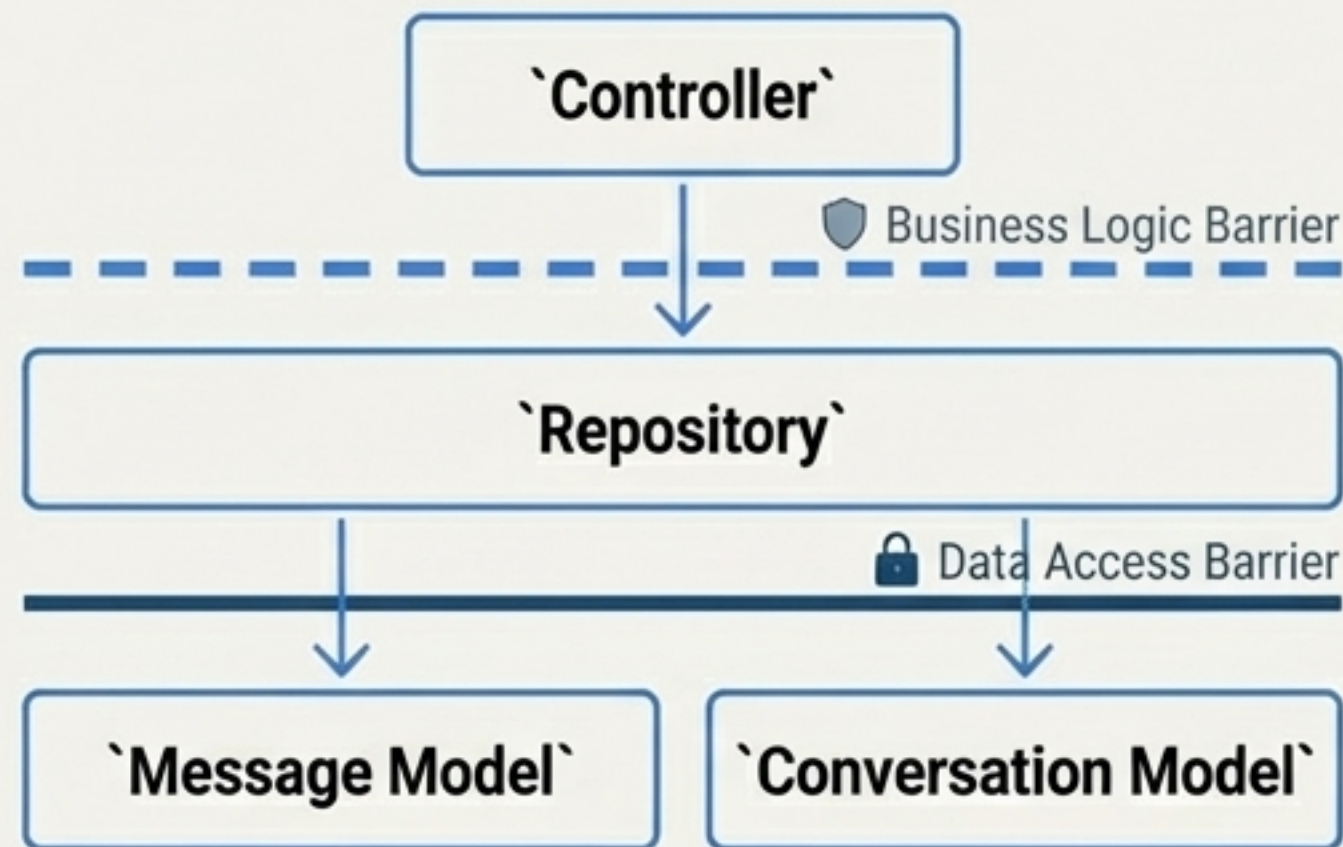
Lớp Truy cập Dữ liệu: `messageRepository`

Lớp này đóng vai trò là cầu nối duy nhất giữa logic ứng dụng và cơ sở dữ liệu. Nó đóng gói tất cả các truy vấn phức tạp, đảm bảo các hoạt động diễn ra một cách an toàn và nhất quán.



Quản lý Hội thoại

- `getConversations(userId)`: Lấy danh sách hội thoại, populate thông tin người dùng và sắp xếp theo ``lastMessageAt``.
- `getOrCreateConversation(userId1, userId2)`: Tìm hoặc tạo một hội thoại mới, đảm bảo không có sự trùng lặp.



Quản lý Trạng thái & Tương tác

- `markConversationAsRead(...)`: Sử dụng ``updateMany`` để đánh dấu tất cả tin nhắn trong một cuộc hội thoại là đã đọc trong một thao tác duy nhất.
- `addReaction(...)`, `removeReaction(...)`, `togglePinMessage(...)`: Quản lý các tương tác phức tạp trên từng tin nhắn.



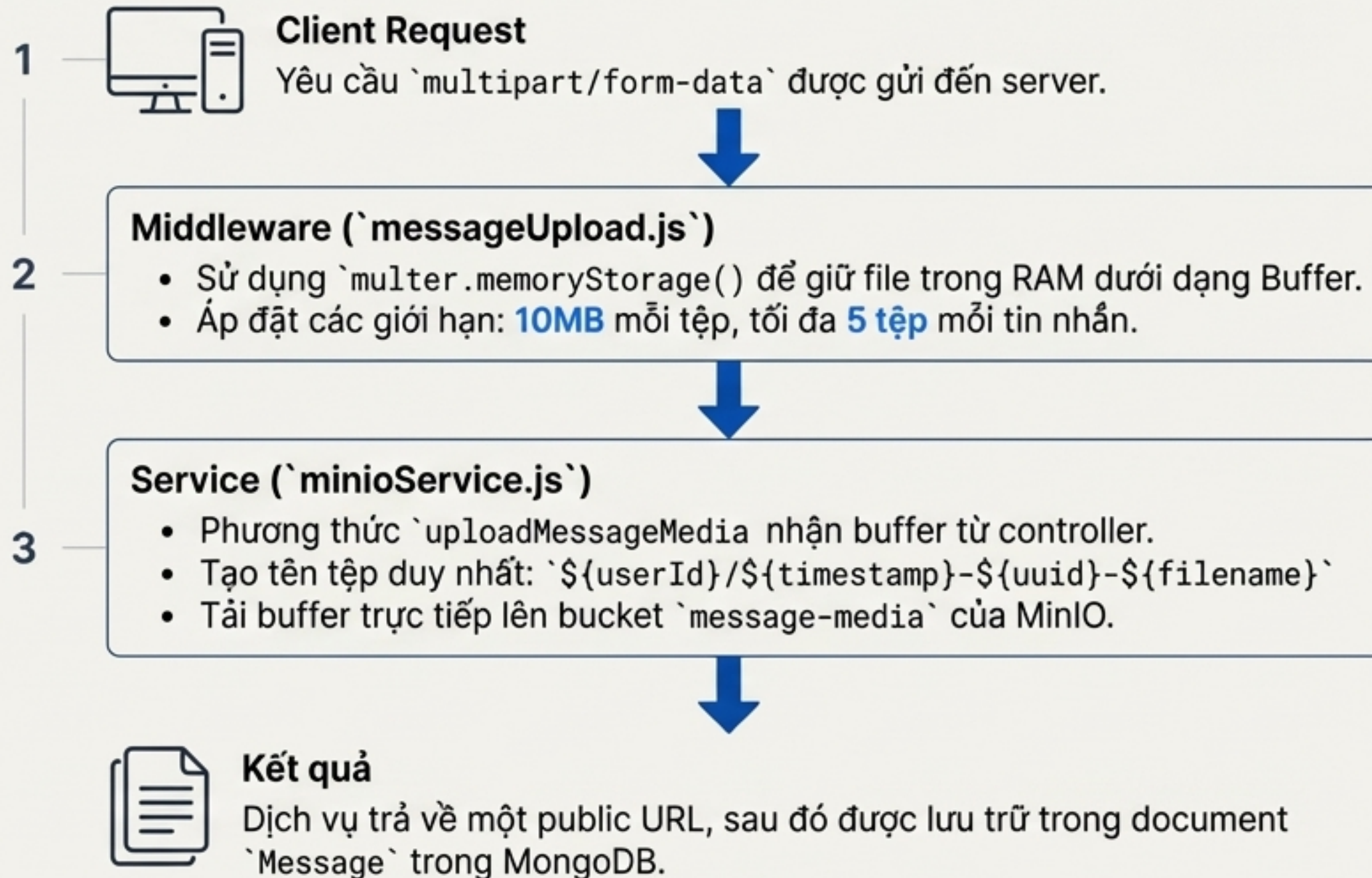
Thao tác Tin nhắn

- `createMessage(...)`: Tạo tin nhắn mới và đồng thời cập nhật trạng thái của ``Conversation`` (`lastMessage`, `unreadCount`) một cách nguyên tử.
- `getMessages(...)`: Lấy tin nhắn trong một hội thoại với hỗ trợ phân trang (`limit/skip`) và loại xóa.
- `deleteMessage(messageId, userId)`: Thực hiện soft delete bằng cách thêm `userId` vào mảng ``deletedBy``.

Xử lý Tệp đính kèm: Luồng `Multer` & `MinIO`

Luồng xử lý file được thiết kế để tối ưu hóa hiệu suất và giảm tải cho server ứng dụng. File không bao giờ được lưu trữ tạm thời trên đĩa của server.

Luồng Xử lý



Ưu điểm Chính



Hiệu suất cao: Không có thao tác ghi/đọc trên đĩa (Disk I/O) của server ứng dụng.



Khả năng mở rộng: Client tải tệp trực tiếp từ MinIO, giảm băng thông cho server ứng dụng.



An toàn: Tên tệp được chuẩn hóa và tổ chức theo namespace của người dùng.

Bộ điều phối (Phần 1): `messageController` và Luồng Tin nhắn Chính

Controller xử lý các yêu cầu HTTP, xác thực đầu vào và điều phối các lớp service và repository. Nó cũng là nơi các sự kiện WebSocket được phát đi để cập nhật client trong thời gian thực.

Các Endpoints Chính



GET /conversations

- Gọi `messageRepository.getConversations` để lấy danh sách hội thoại cho người dùng đã xác thực.



GET /:otherUserId

- Gọi `messageRepository.getMessages`, sử dụng các tham số query `limit` và `skip` để hỗ trợ "infinite scroll".`

Phân tích `POST /send`



1. Xác thực:

- Middleware `authenticateToken` chạy trước.



2. Upload File:

- Middleware `messageUpload` xử lý `req.files`.



3. Xử lý Logic:

- Lặp qua `req.files`, gọi `minioService` cho từng tệp. Xử lý "partial success" (nếu một tệp lỗi, các tệp khác vẫn được xử lý).
- Gọi `messageRepository.createMessage` với nội dung và mảng các tệp đính kèm.



4. Phát sự kiện WebSocket:

- Phát sự kiện **new-message** đến room của người nhận (`user-${receiverId}`).
- Phát sự kiện **new-message** đến room của người gửi (`user-${senderId}`) để đồng bộ hóa giao diện trên các thiết bị. Đây là một bước quan trọng gọi là "Dual Emission".

Bộ điều phối (Phần 2): `messageController` và Các Tính năng Tương tác

Ngoài việc gửi và nhận tin nhắn, controller còn quản lý toàn bộ vòng đời tương tác của người dùng với các cuộc hội thoại.

Luồng Tương tác và Sự kiện Thời gian thực Tương ứng



Đánh dấu đã đọc

PATCH /message/:messageId/read

PATCH /:otherUserId/read

→ Phát `message-read`

→ Phát `conversation-read`



Xóa Tin nhắn

DELETE /:messageId

→ Phát `message-deleted`



Reactions

POST /message/:messageId/reaction

DELETE /message/:messageId/reaction

→ Phát `message-reaction-added`

→ Phát `message-reaction-removed`



Ghim Tin nhắn

PATCH /message/:messageId/pin

→ Phát `message-pinned` hoặc `message-unpinned`



Cài đặt Hội thoại

PATCH /:otherUserId/settings (đổi theme, nickname) → Phát `conversation-settings-updated`

Điểm nhấn

Mỗi hành động của người dùng không chỉ thay đổi dữ liệu trong DB mà còn ngay lập tức thông báo cho các client liên quan thông qua WebSocket, tạo ra một trải nghiệm liền mạch.

Giao diện Hệ thống: `API Routes` & `WebSocket Events`

Đây là bề mặt tương tác của hệ thống. API Routes định nghĩa các hành động có thể được yêu cầu, trong khi **WebSocket Events** mô tả các cập nhật mà server sẽ chủ động đẩy về client.



Định nghĩa API Routes (messageRoutes.js)

```
GET /conversations
GET /:otherUserId
POST /send (với middleware messageUpload)
PATCH /:otherUserId/read
DELETE /:messageId
POST /message/:messageId/reaction
PATCH /message/:messageId/pin
PATCH /:otherUserId/settings
GET /:otherUserId/search
```

Kiến trúc Room

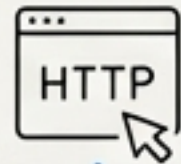
Các sự kiện được nhắm mục tiêu chính xác bằng cách sử dụng các room có tên theo mẫu user- $\{userId\}$.



Các Sự kiện WebSocket Chính (Server → Client)

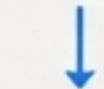
new-message: Gửi toàn bộ đối tượng tin nhắn mới.
message-read: Gửi ID tin nhắn đã được đọc.
conversation-read: Gửi ID hội thoại đã được đọc.
message-reaction-added: Gửi { messageId, reaction: {user, emoji} }.
message-deleted: Gửi ID tin nhắn đã bị xóa.
conversation-settings-updated: Gửi { conversationId, nicknames, themeColor }.

Phân tích Luồng Hoàn chỉnh: Gửi Tin nhắn có Tập đính kèm



1. HTTP POST /send (với `content` và `files`)

Client gửi yêu cầu.



2. Middleware: authenticateToken

Xác thực JWT của người dùng.



3. Middleware: messageUpload.array('files', 5)

Multer phân tích cú pháp yêu cầu, tạo ra `req.files` là một mảng các Buffers trong bộ nhớ.



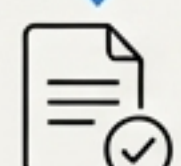
4. Controller: messageController.sendMessage

Xác thực `receiverId` & `content`. Vòng lặp qua `req.files`:
Gọi `minioService.uploadMessageMedia` cho mỗi tập, nhận lại URL công khai.



5. Repository: messageRepository.createMessage

Tạo một document `Message` mới với `content` và mảng `attachments` chứa các URL từ MinIO. Tìm hoặc tạo `Conversation` và cập nhật `lastMessage`, `lastMessageSender`, `lastMessageSender`, `lastMessageAt`, và `unreadCount`.



7. HTTP Response: 201 Created

Server trả về đối tượng tin nhắn vừa được tạo dưới dạng JSON.



6. WebSocket Emission

```
`io.to('user-${receiverId}').emit('new-message', ...)`  
`io.to('user-${senderId}').emit('new-message', ...)`
```


Tối ưu Hiệu năng trên Toàn bộ Hệ thống

Hiệu suất không phải là một tính năng bổ sung, mà là một yêu cầu thiết kế cốt lõi. Các chiến lược tối ưu hóa được áp dụng ở mọi lớp của kiến trúc.



Cấp Cơ sở dữ liệu (Database)

- Sử dụng index phức hợp (`{conversationId, createdAt}`) để tăng tốc độ phân trang và tải tin nhắn.
- Chỉ lấy các trường cần thiết khi populate thông tin người dùng để giảm lượng dữ liệu truyền tải.
- Dùng `updateMany` (ví dụ trong `markConversationAsRead`) để giảm số lượng lệnh gọi đến DB.



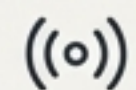
Cấp Lưu trữ Tập (File Storage)

- Client tải tập trực tiếp từ MinIO, giải phóng server ứng dụng khỏi việc proxy tập.
- Kiến trúc URL của MinIO dễ dàng tích hợp với Mạng phân phối nội dung (CDN).



Cấp Ứng dụng (Application)

- API hỗ trợ `limit` và `skip` để tải dữ liệu theo từng phần.
- Tránh ghi file tạm thời lên đĩa, giảm độ trễ I/O.



Cấp Thời gian thực (Real-time)

- Sự kiện chỉ được gửi đến các client thực sự cần chúng, tránh phát sóng không cần thiết.

Các Nguyên tắc Bảo mật và Toàn vẹn Dữ liệu

An toàn và bảo mật được tích hợp vào thiết kế hệ thống, không phải là một lớp được thêm vào sau cùng.

Các Lớp Bảo vệ

Phân quyền (Authorization)



- Mọi hành động (đọc, xóa, ghim) đều xác thực rằng người dùng hiện tại là một thành viên hợp lệ của cuộc hội thoại.
- Chỉ người nhận mới có thể đánh dấu tin nhắn là đã đọc.
- Ngăn chặn rò rỉ dữ liệu giữa các cuộc hội thoại.

Tải lên Tập (File Upload)



- Giới hạn kích thước tập (10MB) và số lượng tập (5) để ngăn chặn các cuộc tấn công từ chối dịch vụ (DoS).
- Tên tập được chuẩn hóa để xử lý các ký tự đặc biệt một cách an toàn.

Quyền riêng tư Dữ liệu (Data Privacy)



- Cơ chế Soft Delete (Roboto Mono) đảm bảo rằng khi một người dùng xóa tin nhắn, nó chỉ biến mất khỏi chế độ xem của họ.
- Yêu cầu xác thực JWT cho tất cả các endpoint.

Bảo mật MinIO



- Bucket được cấu hình để cho phép đọc công khai (public read-only) nhưng chỉ cho phép ghi thông qua backend đã được xác thực.
- Tổ chức tập theo namespace của userId để cô lập dữ liệu.

Chiến lược Xử lý Lỗi và Ghi Log

Một hệ thống mạnh mẽ phải có khả năng xử lý các tình huống không mong muốn một cách linh hoạt và cung cấp đủ thông tin để chẩn đoán sự cố.

Xử lý các Kịch bản Lỗi phổ biến



- **Lỗi tải tệp:**

Chiến lược: "Partial Success". Nếu một trong 5 tệp bị lỗi, tin nhắn vẫn được gửi với các tệp đã tải lên thành công.

✅ Phản hồi: "Hệ thống không bị treo hoàn toàn vì một lỗi nhỏ."



- **ID người nhận không hợp lệ:**

Trả về lỗi **400 Bad Request**.



- **Truy cập không được phép:**

Trả về lỗi **403 Forbidden**.



- **Không tìm thấy tin nhắn:**

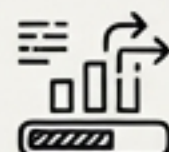
Trả về lỗi **404 Not Found**.



- **Trùng lặp Reaction:**

Trả về lỗi **400** với thông báo rõ ràng.

Chiến lược Ghi Log (Logging)



- **Ghi log tiến trình:**

Theo dõi tiến trình tải lên của từng tệp (ví dụ: **[1/5] Uploading...**).



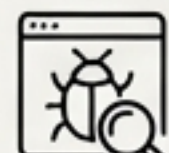
- **Theo dõi Socket Room:**

Ghi log kích thước của các room WebSocket để chẩn đoán các vấn đề kết nối.



- **Cập nhật quan trọng:**

Ghi log các sự kiện tạo/cập nhật hội thoại.



- **Lỗi chi tiết:**

Ghi lại đầy đủ **stack trace** của lỗi để dễ dàng gỡ rối.

Lộ trình Mở rộng và Giám sát

Hệ thống hiện tại được xây dựng vững chắc, nhưng chúng tôi cũng có một kế hoạch rõ ràng để phát triển và đáp ứng nhu cầu ngày càng tăng.



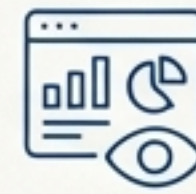
Các Cải tiến về Khả năng Mở rộng (Scalability)

⚠ Hạn chế hiện tại:

- File được xử lý trong bộ nhớ RAM.
- Socket.IO rooms được lưu trữ trên một server duy nhất.
- Chưa có chiến lược lưu trữ tin nhắn cũ.

⚙ Cải tiến Đề xuất:

- Sử dụng **redis adapter** để mở rộng Socket.IO theo chiều ngang.
- Chuyển sang **stream upload** cho các tệp lớn để tránh tiêu thụ RAM.
- Xây dựng chiến lược lưu trữ (**archival**) để di chuyển các tin nhắn cũ.
- Tích hợp CDN với **MinIO** để tăng tốc độ tải tệp.
- Triển khai **quét virus** cho các tệp được tải lên.



Tăng cường Giám sát & Quan sát (Monitoring & Observability)



Logging hiện tại:

- Số lượng tin nhắn, tiến trình upload, kích thước room.

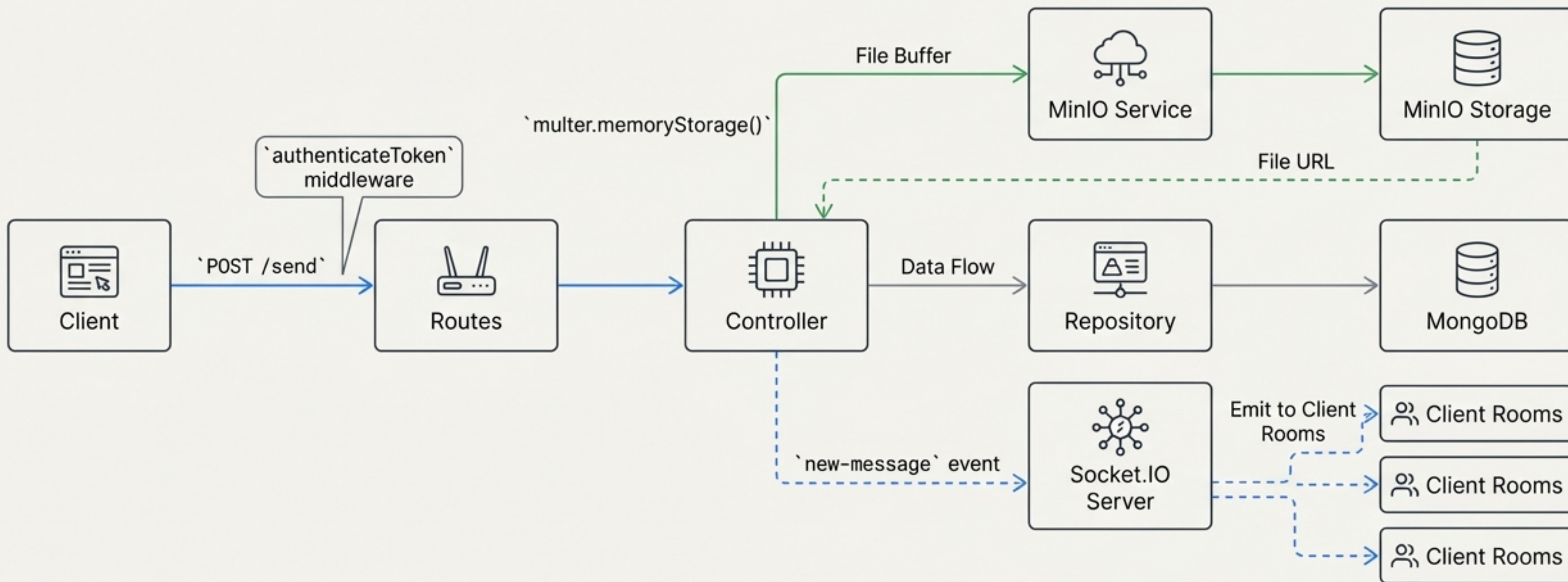


Các chỉ số cần bổ sung:

- Tỷ lệ gửi tin nhắn thành công/thất bại.
- Thời gian tải trung bình của một cuộc hội thoại.
- Số liệu thống kê về tin nhắn chưa đọc.
- Theo dõi tình trạng kết nối WebSocket.
- Dung lượng lưu trữ được sử dụng bởi mỗi người dùng.

Sơ đồ Kiến trúc Toàn cảnh: Hệ sinh thái Tin nhắn 1-1

Một cái nhìn tổng quan về cách các thành phần tương tác để cung cấp trải nghiệm nhắn tin thời gian thực mạnh mẽ.



Kiến trúc này cân bằng giữa hiệu suất, khả năng mở rộng và sự mạnh mẽ, tạo nền tảng vững chắc cho trải nghiệm giao tiếp thời gian thực chất lượng cao.